Physics-Informed Learning of Feedback-Linearizing Representations

Pavlos Kallinikidis*[1], Fengjun Yang*[1], David Snyder[1], Jacob H. Seidman[2], Nikolai Matni[1], Paris Perdikaris[3], and George J. Pappas[1]

***Abstract*— Feedback linearization is a powerful tool in nonlinear control, but finding the linearizing coordinate transformation remains challenging. Feedback linearizing transformations are governed by well-established partial differential equations (PDEs) whose well-posedness is based on the Lie-algebraic Frobenius theorem. Even if the conditions of the Frobenius theorem are satisfied, solving the PDEs becomes computationally intractable for large system dimensions. To address this challenge, we propose a cascaded physics-informed neural network (PINN) framework to approximately solve these PDEs. By separately parameterizing terms of different Lie derivative orders, we mitigate the compounding errors inherent in taking high-order Lie derivatives of neural networks. Furthermore, we use the learned transformation to design a tracking controller and establish theoretical bounds on its error with respect to inaccuracies in the learned feedback-linearizing representation. We validate our method on a broad class of feedback linearizable systems, including a multi-input, multi-output planar quadrotor and synthetic examples for which analytical approaches are impractical, and demonstrate that our approach can computationally discover effective feedback-linearizing representations of nonlinear systems for control tasks.**

I. Introduction

Feedback linearization transforms nonlinear dynamics into equivalent linear ones via feedback transformation [1]. Once feedback-linearizing coordinates have been found, this transformation enables the application of powerful tools from linear systems theory to otherwise nonlinear control systems. The existence of such transformations depends on a Lie-algebraic Frobenius condition. When the Frobenius condition holds, there exists a well-posed system of partial differential equations whose solution and its Lie derivatives provide us with the linearizing coordinates. However, solving them is challenging: analytical approaches are customized and highly system-specific, while algorithmic approaches are scarce and suffer from significant computational shortcomings [2].

*Equal contribution. [1]GRASP Lab, University of Pennsylvania, Philadelphia, PA, USA. [2]Reality Defender Inc. [3]Department of Mechanical Engineering and Applied Mechanics, University of Pennsylvania, Philadelphia, PA, USA. This work was supported in part by DARPA TRS program under contract HR00112590145.

Recently, PINNs [3] have emerged as an efficient method to approximate PDE solutions with applications to, for example, optimal control [4] and reachability [5]. PINNs recast PDE solutions as minimizers of a residual loss. As such, their computational cost empirically scales with solution complexity rather than grid resolution [5], thereby overcoming the computational challenges for higher-dimensional problems.

We leverage the computational benefits of PINNs to systematically discover feedback-linearizing representations of nonlinear systems. Two key technical challenges arise. First, during learning, the high-order Lie derivatives in the governing PDEs introduce computational bottlenecks and compounding approximation errors. To overcome this, we introduce a cascaded neural network structure that decomposes complex, high-order PDEs into a sequence of tractable, first-order residual minimizations. We demonstrate that this approach significantly outperforms naive PINN baselines and accurately reconstructs linearizing behavior. Second, controller design must account for the effect of a learned (approximate) representation on downstream control performance. We design a controller using the learned transformation and bound its downstream tracking error with respect to learning errors. Evaluated on both physical and synthetic systems up to eight dimensions, our work constitutes the first systematic application of PINNs for discovering feedback linearizing representations.

A. Related Work

1) Learning Linear Representations of Nonlinear Systems: Prior learning-based approaches for feedback linearization focus primarily on compensating for uncertain dynamics when the linearizing outputs are already known, whether by directly approximating the linearizing control law [6], [7], [8], [9] or modeling the dynamic mismatch from an imperfect linearization [10], [11]. In contrast, we consider the inverse problem of discovering linearizing outputs when the dynamics are perfectly known. Separately, Goswami et al. [12] aim to discover linearizing transformations via purely data-driven invertible neural networks, which limits downstream control analysis. By contrast, our approach is model-

based, solving the governing PDE conditions to discover the desired linear representation while enabling formal tracking error bounds. Other linearizing frameworks, such as Koopman operator approaches [13], [14], [15], focus largely on system dynamics and do not leverage control-oriented structure like the Frobenius condition.

2) Physics-Informed Neural Networks in Control: By casting PDE solving as optimization over sampled points, PINNs [3] have been successfully applied to control tasks involving Hamilton-Jacobi-Bellman [4] and Hamilton-Jacobi-reachability equations [5]. However, their application to feedback linearization remains unexplored. A primary barrier is that naively applying PINNs to the high-order PDEs that characterize linearizing transformations introduces significant computational bottlenecks and numerical inaccuracies—challenges we overcome in this work through a novel cascadic training algorithm.

B. Statement of Contributions

Our contributions are threefold. We present:

- A novel, cascaded physics-informed learning framework that efficiently approximates linearizing transformations for both single-input single-output (SISO) and multiple-input multiple-output (MIMO) systems;
- A realizable controller with learning-aware tracking error bounds that depend explicitly on the quality of the learned coordinate transform, directly linking the learning effectiveness to downstream control performance;
- Extensive empirical validation demonstrating our method's capacity to effectively feedback-linearize a broad class of complex dynamical systems, including a planar quadrotor model and systems for which analytical solutions are intractable.

C. Notations

We use $D_{\mathbf{x}}f$ to denote the derivative of f with respect to $\mathbf{x}$. When f and $\mathbf{x}$ are both vectors, $D_{\mathbf{x}}f$ denotes the Jacobian. We use the notation $\mathcal{L}_f h(\mathbf{x}) := D_{\mathbf{x}}h(\mathbf{x}) \cdot f(\mathbf{x})$ to denote the Lie derivative of h along the vector field f. We use $\mathrm{ad}_f g := [f, g] = D_{\mathbf{x}}g \cdot f - D_{\mathbf{x}}f \cdot g$ to denote the Lie bracket between vector fields f and g. For convenience, we denote the iterative Lie brackets as $\mathrm{ad}_f^{k+1} g := [f, \mathrm{ad}_f^k g]$ with $\mathrm{ad}_f^0 g = g$.

II. Problem Formulation

Consider the nonlinear, MIMO, control-affine system

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \tag{1}$$

where state $\mathbf{x} \in \mathbb{R}^n$, input $\mathbf{u} \in \mathbb{R}^m$, and $f : \mathbb{R}^n \to \mathbb{R}^n, g : \mathbb{R}^n \to \mathbb{R}^{n\times m}$ are smooth vector fields. Given *perfect knowledge* of f and g over an open region $\mathcal{D} \subseteq \mathbb{R}^n$, our objective is to *learn* a diffeomorphism $\Phi : \mathcal{D} \to \mathbb{R}^n$ and a nonlinear feedback control law $\mathbf{u} = \psi(\mathbf{x}, \mathbf{v})$ such that the transformed state $\mathbf{z} = \Phi(\mathbf{x})$ follows linear, time-invariant dynamics

$$\dot{\mathbf{z}} = \mathbf{A}\mathbf{z} + \mathbf{B}\mathbf{v}, \tag{2}$$

where $(\mathbf{A}, \mathbf{B})$ is a canonical Brunovsky pair (i.e. a decoupled chain of integrators).

Henceforth, we assume that such a diffeomorphism $\Phi(\mathbf{x})$ exists and refer to it as the *linearizing coordinate transform*.

In what follows, we first recall key properties for feedback linearization in Section III. Building on these properties, we propose a physics-informed learning framework in Section IV. In Section V, we illustrate how the learning error affects downstream tracking performance. Finally, we present extensive experimental results in Section VI.

III. Background on Feedback Linearization

The theory of feedback linearization yields a set of PDEs whose solution directly determines the linearizing coordinate transform Φ. To maintain notational clarity, we restrict our primary exposition to the SISO setting ($m = 1$, where $g(\mathbf{x})$ is a single vector field) and discuss the MIMO generalization at the end of this section. In the SISO setting, the existence of a linearizing coordinate transformation is equivalent to that of a smooth, real-valued *linearizing output* $h(\mathbf{x})$ that has relative degree n on the domain $\mathcal{D}$ (see, e.g., Lemma 4.2.1 in [1]), i.e., for all $\mathbf{x} \in \mathcal{D}$, $h(\mathbf{x})$ satisfies:

CA1 $\mathcal{L}_g \mathcal{L}_f^i h(\mathbf{x}) = 0$ for $i \in \{0, \dots, n-2\}$,
CA2 $\mathcal{L}_g \mathcal{L}_f^{n-1} h(\mathbf{x}) \neq 0$.

The linearizing transformation can then be constructed directly by stacking $h(\mathbf{x})$ and its Lie derivatives. Specifically, $\Phi(\mathbf{x}) = [\Phi_1(\mathbf{x}), \dots, \Phi_n(\mathbf{x})]^\top$, where

$$\Phi_i(\mathbf{x}) = \mathcal{L}_f^{i-1} h(\mathbf{x}). \tag{3}$$

The corresponding linearizing control law is given as

$$\mathbf{u} = \psi(\mathbf{x}, \mathbf{v}) = A_\Phi(\mathbf{x})^{-1}(\mathbf{v} - b_\Phi(\mathbf{x})), \tag{4}$$

where $A_\Phi(\mathbf{x}) = \mathcal{L}_g \Phi_n(\mathbf{x})$ and $b_\Phi(\mathbf{x}) = \mathcal{L}_f \Phi_n(\mathbf{x})$. Consequently, **CA** readily defines a PDE whose solution fully parameterizes the desired transformation Φ. However, computing this solution is challenging due to the presence of high-order Lie derivatives. To reduce the order, the Lie derivatives are recast in terms of iterated Lie brackets to get the following set of equivalent, first-order conditions:

CB1 $\mathcal{L}_{\mathrm{ad}_f^i g} h(\mathbf{x}) = 0$ for $i \in \{0, \dots, n-2\}$,

CB2 $\mathcal{L}_{\mathrm{ad}_f^{n-1}g}h(\mathbf{x}) = (-1)^{n-1}\mathcal{L}_g\mathcal{L}_f^{n-1}h \neq 0.$

The condition (CB1) constitutes a set of $n-1$ equalities

$$D_{\mathbf{x}}h \cdot \begin{bmatrix} g & \mathrm{ad}_f g & \dots & \mathrm{ad}_f^{n-2}g \end{bmatrix} = 0 \tag{5}$$

that (as developed in Section IV) are well-suited for supervised regression. However, the non-triviality condition (CB2) ensuring the numerical stability of the control law (4) admits multiple implementations. A natural approach, in line with the form of (CB1), is to enforce

$$\mathcal{L}_{\mathrm{ad}_f^{n-1}g}h(\mathbf{x}) = c \tag{6}$$

for a nonzero constant c over the domain $\mathcal{D}$. However, this equality formulation is restrictive; a solution can fail to exist for (6) even when one exists for **CB**. Hence, in Section IV we propose a more general method to reformulate the non-triviality condition.

Finally, **CA** and **CB** extend naturally to the MIMO setting, where a function $h(\mathbf{x})$ has relative degree r_i if

CC1 $\mathcal{L}_{g_j}\mathcal{L}_f^i h(\mathbf{x}) = 0$ for $i \in \{0, \dots, r_i - 2\}$ and $j \in \{1, \dots, m\}$,

CC2 $\mathcal{L}_{g_j}\mathcal{L}_f^{r_i-1}h(\mathbf{x}) \neq 0$ for any $j \in \{1, \dots, m\}$,

for all $\mathbf{x} \in \mathcal{D}$. A MIMO system is feedback linearizable if there exists a set of real-valued smooth output functions $\{h_i(\mathbf{x})\}_{i=1}^m$ whose respective relative degrees r_i add up to n, i.e. $\sum_{i=1}^m r_i = n$. The linearizing control law retains the form of (4) with $A(\mathbf{x})$ and $b(\mathbf{x})$ generalized to matrix- and vector-valued functions that depend on the highest-order Lie derivatives (i.e. $\mathcal{L}_f^{r_i}h_i$, $\mathcal{L}_{g_j}\mathcal{L}_f^{r_i-1}h_i$). When $A(\mathbf{x})$ is invertible, the full diffeomorphism $\Phi(\mathbf{x})$ can be constructed by stacking these outputs and their successive Lie derivatives. We defer further details to Appendix A of the full version [16]. Because of the shared structure between SISO and MIMO feedback linearization, the approach we present applies naturally to both settings.

IV. Learning Linearizing Transformations

We propose a PINN framework to compute the desired diffeomorphism Φ by approximating solutions to the aforementioned PDEs. We initially present a direct Lie bracket-based algorithm for learning the linearizing output. We then show that deriving the full diffeomorphism through high-order differentiation of this output introduces severe compounding errors in the learning process. To address this, we introduce a novel cascaded neural network structure that sequentially minimizes tractable, first-order residuals to mitigate these errors and enable effective downstream control.

A. Learning Linearizing Outputs

PINNs [3] compute approximate PDE solutions by parameterizing the candidate function as a deep neural network and minimizing the PDE residuals via a standard loss function. Defining the candidate linearizing output $h_\theta(\mathbf{x})$ with network weights θ, the consequent PDE residuals of (CB1) are compactly expressed as

$$\mathcal{RB}_\theta(\mathbf{x}) := D_{\mathbf{x}}h_\theta \cdot \begin{bmatrix} g & \mathrm{ad}_f g & \dots & \mathrm{ad}_f^{n-2}g \end{bmatrix},$$

while the non-triviality residual of (CB2) as

$$\mathcal{SB}_\theta(\mathbf{x}) := D_{\mathbf{x}}h_\theta \cdot \mathrm{ad}_f^{n-1}g.$$

We appeal to the first-order Lie bracket-based conditions **CB**, to circumvent the computational complexity introduced by the high order Lie derivatives in **CA**. Since $\mathcal{D} \subset \mathbb{R}^n$ is compact and $\mathcal{SB}(\mathbf{x}) = D_{\mathbf{x}}h \cdot \mathrm{ad}_f^{n-1}g$ is continuous for any continuously differentiable $h(\mathbf{x})$, we can always pick $a, b > 0$ and find a linearizing output s.t. $\mathcal{SB}(\mathbf{x}) \in [a, b]$ for all $\mathbf{x} \in \mathcal{D}$, ensuring non-triviality while preserving regularity (e.g., to avoid excessively large control gains). We can now train the network $h_\theta(x)$ to approximate such an output using the composite loss

$$L(\theta) = L_{\mathcal{RB}}(\theta) + L_{\mathcal{SB}}(\theta). \tag{7}$$

with $L_{\mathcal{RB}}(\theta) = \sum_{i=1}^N ||\mathcal{RB}_\theta(x_i)||_2^2$ to enforce the desired bracket annihilation and $L_{\mathcal{SB}}(\theta) = \sum_{i=1}^N (\max(a - \mathcal{SB}_\theta(\mathbf{x}_i), 0) + \max(\mathcal{SB}_\theta(\mathbf{x}_i) - \beta, 0))^2$ to impose more general box constraints preventing triviality.

B. Difficulty in Constructing Φ via Differentiation

While the loss function (7) serves as an effective supervision for finding linearizing outputs, constructing the diffeomorphism Φ requires the complete set of Lie derivatives that appear in (CA1)–(CA2). However, we observe that a neural network trained under loss (7) can only approximate a true linearizing output well up to its first derivative; the highest-order derivative that is explicitly supervised. This limitation is closely related to the deep learning phenomenon of *spectral bias* [17], which describes the tendency of neural networks to preferentially approximate low-frequency components of a target function. This bias is detrimental to the approximation of higher order Lie derivatives, since differentiation amplifies errors proportionally to frequency.

Additionally, the small first-order bracket errors (7) in the learned outputs can be amplified via high-order differentiation, as illustrated in the following lemma.

Lemma 1. *Let* $e_i(\mathbf{x}) := \mathcal{L}_{\mathrm{ad}_f^i g}h(\mathbf{x})$*, for* $\mathbf{x} \in \mathcal{D}$*, be the residuals of* (CA1)*. We also define the residuals in*

(CB1) *as* $\epsilon_i(\mathbf{x}) := \mathcal{L}_g\mathcal{L}_f^i h(\mathbf{x})$. *Assuming known* e_i *for* $i \in \{0, \dots, k\}$, ϵ_i *is computed with the iterative formula:*

$$\epsilon_i = (-1)^i e_i + \sum_{r=0}^{i-1}(-1)^{r+i+1}\binom{i}{r}\mathcal{L}_f\epsilon_r.$$

A direct consequence of Lemma 4.1.2 in [1], this result formalizes the attenuation of implicit supervision over (CA1) residuals e_i when training on (CB1), as the minimization targets ϵ_i come to be dominated by the accumulating right-most terms. Thus, the direct approach only weakly penalizes violations of higher-order Lie derivative conditions, compromising the reliability of the learned transform in downstream control tasks.

To mitigate this, one could augment the loss in (7) with explicit supervision of higher order Lie derivatives via the conditions **CA**. However, incorporating high-order Lie derivatives introduces significant computational and optimization challenges. Although the former can be addressed to some extent by Taylor-mode automatic differentiation [18], the latter make this approach impractical as system complexity grows (see Section VI).

C. Cascadic Learning of the Diffeomorphism

To overcome the limitations of direct bracket-based supervision, we adopt a cascaded representation using separate neural networks

$$\hat{\Phi}(\mathbf{x}) := \left(h_{\theta_1}(\mathbf{x}), \dots, h_{\theta_n}(\mathbf{x})\right), \tag{8}$$

parametrizing the linearizing coordinate transform $\hat{\Phi}(\mathbf{x}) = \left(\hat{\Phi}_1(\mathbf{x}), \dots, \hat{\Phi}_n(\mathbf{x})\right)$. Exploiting the ability of the bracket-based approach to approximate a linearizing output up to its first order, we combine this cascaded structure with a curriculum training strategy. In particular, we first train $h_{\theta_1}(\mathbf{x})$ using (7). Then, for $i = 2, \dots, n$, we train each $h_{\theta_i}(\mathbf{x})$ in succession by minimizing

$$L(\theta_i) = L_{\mathcal{F}}(\theta_i) + L_{\mathcal{G}}(\theta_i), \tag{9}$$

where the consistency term

$$L_{\mathcal{F}}(\theta_i) = \sum_{j=1}^{N}\left(\mathcal{L}_f h_{\theta_{i-1}}(\mathbf{x}_j) - h_{\theta_i}(\mathbf{x}_j)\right)^2 \tag{10}$$

encourages reconstruction of the Lie derivative of the preceding layer $h_{\theta_{i-1}}$, while the control-specific term

$$L_{\mathcal{G}}(\theta_i) = \sum_{j=1}^{N}\left(\mathcal{L}_g h_{\theta_i}(\mathbf{x}_j)\right)^2 \tag{11}$$

promotes adherence to conditions **CA**.

Intuitively, the cascadic approach decomposes the original learning problem—where a single network must satisfy conditions involving Lie derivatives up to order n—into a sequence of simpler tasks involving only first-order derivatives across multiple networks. By relaxing the implicit constraint that $\hat{\Phi}_{i+1}(\mathbf{x}) = \mathcal{L}_f\hat{\Phi}_i(\mathbf{x})$, the resulting coordinate transformation satisfies **CA** more tightly than approaches relying solely on bracket-based supervision. Moreover, this formulation avoids the optimization pathologies associated with explicitly supervising high-order Lie derivatives. Empirically, the cascadic approach demonstrates superior performance compared to single-network formulations, as evidenced by experiments on multiple complex systems.

V. Tracking Control with Learned Coordinate Transform

To illustrate how learning errors manifest in downstream control performance, we consider the task of following a dynamically feasible and bounded reference trajectory $(\bar{\mathbf{x}}, \bar{\mathbf{u}})$ under the nonlinear dynamics (1). With the learned, approximately linearizing, transform $\hat{\Phi}$, we design an error-feedback controller in the linearized space. We characterize the tracking error of this controller in terms of learning losses. Before proceeding, we make the following assumptions on the map $\hat{\Phi}$.

Assumption 1. $\hat{\Phi}$ is invertible. Further, there exists a positive constant $l_{\hat{\Phi}}$ such that $l_{\hat{\Phi}}\|\mathbf{x}_1 - \mathbf{x}_2\| \leq \left\|\hat{\Phi}(\mathbf{x}_1) - \hat{\Phi}(\mathbf{x}_2)\right\|$ for all $\mathbf{x} \in \mathcal{D}$.

The invertibility of $\hat{\Phi}$ is necessary to relate the tracking error in the linearized domain to that in the nonlinear domain and is a standard assumption for analyzing approximate linearizing transformations [19]. The existence of $l_{\hat{\Phi}}$ ensures that the inverse is not arbitrarily ill-conditioned.

Let $\bar{\mathbf{z}} = \hat{\Phi}(\bar{\mathbf{x}})$ be the reference linearized state and $\bar{\mathbf{v}} := \mathcal{L}_f\hat{\Phi}_{n-1}(\bar{\mathbf{x}}) + \mathcal{L}_g\hat{\Phi}_{n-1}(\bar{\mathbf{x}})\bar{\mathbf{u}}$ as the reference virtual input. We define the control law in the linearized space:

$$\mathbf{v}(t) = \bar{\mathbf{v}}(t) - \mathbf{K}(\mathbf{z} - \bar{\mathbf{z}}(t)),$$

where $\mathbf{K}$ is a robust stabilizing gain matrix designed by solving the $\mathcal{H}_\infty$ Riccati equation:

$$\mathbf{PA} + \mathbf{A}^\top\mathbf{P} - \mathbf{PBR}^{-1}\mathbf{B}^\top\mathbf{P} = -\mathbf{Q} - \frac{1}{\gamma^2}\mathbf{PP}. \tag{12}$$

Here, $(\mathbf{A}, \mathbf{B})$ is the Brunovsky pair, and $\mathbf{Q}$, $\mathbf{R}$ are properly chosen cost matrices. The gain matrix is then taken to be $\mathbf{K} = \mathbf{R}^{-1}\mathbf{BP}$. Applying the linearizing control law (4), the control input is given as

$$\mathbf{u} = A_{\hat{\Phi}}^{-1}(\mathbf{x})\left(\bar{v} - \mathbf{K}\left(\mathbf{z} - \bar{\mathbf{z}}\right) - b_{\hat{\Phi}}(\mathbf{x})\right). \tag{13}$$

Because of the existence of learning errors, $\hat{\Phi}$ does not perfectly linearize the $\mathbf{z}$-space dynamics and leads to the

following *perturbed* $\mathbf{z}$-dynamics

$$\dot{\mathbf{z}} = \mathbf{A}\mathbf{z} + (\mathbf{B} + \Delta_B(\mathbf{x}))\mathbf{v} + \mathbf{d}_z(\mathbf{x}), \tag{14}$$

where the multiplicative input disturbance $\Delta_B(\mathbf{x})$ and the drift $\mathbf{d}_z(\mathbf{x})$ arise from $\hat{\Phi}$ not perfectly satisfying conditions **CA**. Two sources of learning inaccuracy drive these terms: the PDE residuals $r_i(\mathbf{x}) := \mathcal{L}_g\hat{\Phi}_i(\mathbf{x})$, and the cascade consistency errors $\mathbf{d}_c(\mathbf{x})$, with i-th element $\mathbf{d}_{c,i}(\mathbf{x}) := \mathcal{L}_f\hat{\Phi}_{i-1}(\mathbf{x}) - \hat{\Phi}_i(\mathbf{x})$. Denoting the stacked residual $\mathbf{r}(\mathbf{x}) := [r_1(\mathbf{x}), \cdots, r_{n-2}(\mathbf{x}), 0]$, the perturbation terms can be expressed as

$$\begin{aligned} \Delta_B(\mathbf{x}) &:= \mathbf{r}(\mathbf{x})A_{\hat{\Phi}}^{-1}(\mathbf{x}), \\ \mathbf{d}_z(\mathbf{x}) &:= \mathbf{r}(\mathbf{x})A_{\hat{\Phi}}^{-1}(\mathbf{x})b_{\hat{\Phi}}(\mathbf{x}) + \mathbf{d}_c(\mathbf{x}). \end{aligned} \tag{15}$$

Note that the PDE residuals $\mathbf{r}$ and consistency error $\mathbf{d}_c$ are explicitly minimized during training via corresponding loss terms in (9). Further, given that $A_{\hat{\Phi}}^{-1}(\mathbf{x})$ is invertible (promoted via the term $L_{\mathcal{S}}$ in (7)), $\|\Delta_B(\mathbf{x})\|_F$ and $\|\mathbf{d}_z(\mathbf{x})\|_2$ vanish as $\mathbf{r}$ and $\mathbf{d}_c$ approach zero. Thus, when the learning error is small, it is reasonable to assume that the disturbance terms (15) are small and bounded over the training domain.

Assumption 2. We assume that for all $\mathbf{x} \in \mathcal{D}$, the disturbance terms $\|\Delta_B(\mathbf{x})\|_F \leq \bar{\delta}$, $\|\mathbf{d}_z(\mathbf{x})\|_2 \leq \bar{d}$.

Finally, to ensure safety, the reference trajectory must maintain a sufficient margin strictly within the domain where our learning guarantees hold.

Assumption 3. The reference virtual input is bounded by $\|\bar{\mathbf{v}}(t)\| \leq \bar{V}$, and the reference states $\mathbf{x}(t)$ remain strictly inside the interior of the training domain $\mathcal{D}$.

In the interest of space, we present an informal version of the result, deferring the formal statement and proof to Appendix B of the full version [16].

Theorem 2 (informal)**.** *Suppose Assumptions 1–3 hold and that the input perturbation is sufficiently small such that $\bar{\delta} < \sqrt{\lambda_{min}(\mathbf{R})}/\gamma$. Then, there exists a forward-invariant set $\Omega_x = \{\mathbf{e} \mid \|\mathbf{e}\|_2^2 \leq r_{\mathbf{e}}^2\}$ for the state tracking error $\mathbf{e} = \mathbf{x} - \bar{\mathbf{x}}$. The radius $r_{\mathbf{e}}$ is bounded by the additive and multiplicative learning errors and scales as:*

$$r_{\mathbf{e}}^2 = \mathcal{O}\left(\frac{\bar{d}^2 + \bar{\delta}^2\bar{V}^2}{\lambda_{min}(\mathbf{R}) - \gamma^2\bar{\delta}^2}\right). \tag{16}$$

Consequently, if the initial state satisfies $\mathbf{e}_x(0) \in \Omega_x$, and the reference trajectory maintains a safety margin from the boundary of $\mathcal{D}$ such that $\bar{\mathbf{x}}(t) \in \mathcal{D} \ominus \Omega_x$, the true state $\mathbf{x}(t)$ never leaves $\mathcal{D}$, guaranteeing that the local error bounds hold for all $t \geq 0$.

Theorem 2 formalizes the relationship between the learning errors and closed-loop control. Driving the PDE residuals $\mathbf{r}(x)$ and consistency errors $\mathbf{d}_c(\mathbf{x})$ to zero during training directly shrinks $\bar{\delta}$ and $\bar{d}$, thereby tightening the guarantees on downstream tracking performance. In Section VI, we empirically evaluate this bound by estimating the error bounds $\bar{\delta}$ and $\bar{d}$ via sampling.

VI. Experiments

We test our learning algorithm and the corresponding learned downstream controllers in an extensive array of feedback linearizable systems. We first test our method on well-known examples, whose linearizing outputs are often derived by hand and thus have simple forms that are easy to learn. Then, to further stress-test our approach, we synthesize complex SISO and MIMO systems for which hand derivations are intractable. The results show that our approach can achieve good empirical performance for systems of various sizes in terms of both the quality of the linearizing transform and the controller performance.

A. Learning Setup and Evaluation

We employ Jaxpi [20] as our PINN implementation framework and adopt the modified MLP architecture proposed in [21] for our candidate PDE solutions, as it exhibits superior performance compared to standard feed-forward MLPs. Specifically, we parameterize the modified MLPs with three hidden layers of 256 neurons and GELU activation functions. The network weights are initialized using the scheme suggested in [22] and training is performed using the ADAM optimizer for 200–300 $\times 10^3$ iterations for the first and 150–200 $\times 10^3$ for the subsequent layers of the cascade.

In addition to our **cascadic** approach, we also implement the **direct** single-network approach that learns a linearizing output with supervision from both conditions **CA** and **CB** and constructs the diffeomorphism directly from its Lie derivatives.

In addition to reporting the metrics explicitly included in the losses supervising training, we seek to directly measure how well the learned coordinate transform linearizes the system dynamics. To do so, we sample $\mathbf{x}$ uniformly in $\mathcal{D}$, $\mathbf{u} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and compute the difference between the true time derivative of the transform $\dot{\hat{\Phi}}(\mathbf{x}) = D_{\mathbf{x}}\hat{\Phi}(\mathbf{x})(f(\mathbf{x}) + g(\mathbf{x})\mathbf{u})$ and the time derivative predicted by the linear dynamics $\dot{\hat{\Phi}}^{pred}(\mathbf{x}) = \mathbf{A}\hat{\Phi}(\mathbf{x}) + \mathbf{B}(\mathcal{L}_f\hat{\Phi}_{n-1}(\mathbf{x}) + \mathcal{L}_g\hat{\Phi}_{n-1}(\mathbf{x})\mathbf{u})$. We then measure the quality of linearization as the relative error of the true and approximate time derivatives:

$$\varepsilon(\mathbf{x}) := \left\|\dot{\hat{\Phi}}(\mathbf{x}) - \dot{\hat{\Phi}}^{pred}(\mathbf{x})\right\| \Big/ \left\|\dot{\hat{\Phi}}(\mathbf{x})\right\|. \tag{RLQE}$$

This *relative linearization quality error* (RLQE) is especially useful for comparing our proposed cascadic

approach and the single network baseline, as the former intentionally introduces a cascade inconsistency between $T_{i+1}(\mathbf{x})$ and $\mathcal{L}_f T_i(\mathbf{x})$ but the latter does not, making their PDE residual not directly comparable. In contrast, RLQE provides a fair comparison as it directly quantifies the quality of the downstream linearization.

B. Controller Setup and Evaluation

Once the linearizing coordinate transformation is learned, we construct the downstream tracking controller following the procedure in Section V with $\mathbf{Q} := q\mathbf{I}$ and $\mathbf{R} := \mathbf{I}$, where $q \in [1, 5]$ is system-specific. To evaluate the controller, for each system, we generate 64 reference trajectories by using a ground-truth feedback linearization controller to regulate the state to the origin from initial conditions randomly sampled from the interior of the domain. Example reference trajectories are visualized in Figures 1 and 2. To test robustness, we apply the tracking controller from perturbed initial conditions, with perturbations sampled from an ℓ_∞ ball of radius 0.15. We simulate the controllers for $T \in [8, 10]$ seconds with RK45 numerical integration and a step size of 0.002s.

We evaluate the tracking controller with the *average tracking error* ($\frac{1}{T}\int_0^T \|\mathbf{x} - \bar{\mathbf{x}}\| \, \mathrm{d}t$), the *final state error* ($\|\mathbf{x}(T) - \bar{\mathbf{x}}(T)\|$), and the *total control effort* ($\int_0^T \|\mathbf{u}\| \, \mathrm{d}t$). For each system, we report the median errors among the 64 reference trajectories, along with the number of diverging trajectories, to capture potential stability issues. To contextualize the results, we also report the performance of a feedback linearization controller constructed with the ground-truth diffeomorphism.

C. Results: Real Systems

We first test our pipeline on two textbook examples with affine linearizing outputs: a single-link manipulator with a flexible joint (see Example 6.10 in [23]) and a planar quadrotor.

1) Flexible Joint: The single link manipulator is a fourth order SISO system with a linearizing output $h(\mathbf{x}) = x_1$ (see Example 6.10 in [23]). We try the single-network approach of Section IV-B and observe that it is sufficient for the minimization of the composite loss, consisting of both bracket and high-order Lie derivative terms. The residuals of conditions **CA** for the learned linearization output are summarized in Table I and we note that they indeed result in negligible RLQE.

$\lvert L_g h\rvert$	$\lvert L_g L_f h\rvert$	$\lvert L_g L_f^2 h\rvert$	$L_g L_f^3 h$	RLQE
4.3×10^{-7}	1.6×10^{-7}	9.1×10^{-7}	1.101	$\mathbf{1.4 \times 10^{-8}}$

TABLE I: Mean Lie derivatives and RLQE for the learned output for the direct method in the flexible joint example.

For the tracking task, we report the performance of both learned and ground truth transforms in the domain $\mathcal{D} = [-\pi, \pi] \times [-5, 5] \times [-\pi, \pi] \times [-5, 5]$ in Table II. The learned linearizing outputs demonstrate robustness to small numerical training errors, captured by the RLQE, achieving nearly identical performance metrics as the ground truth controller.

Metric	Ground Truth	Direct	Cascadic
RLQE	—	1.4×10^{-8}	5.4×10^{-7}
Avg. error	1.000	1.003	1.010
Final error	0.004	0.004	0.004
Control effort	605.234	605.333	603.026

TABLE II: Tracking performance of the nominal and learned transforms in the single link manipulator system.

2) Planar Quadrotor: Another system known to be differentially flat and, therefore, dynamically feedback linearizable is the planar quadrotor in extended dynamics form, represented by the state equations

$$\dot{p}_1 = v_1; \quad \dot{v}_1 = -\frac{F}{m}\sin(\theta); \qquad \dot{F} = \nu; \quad \dot{\nu} = u_1$$
$$\dot{p}_2 = v_2; \quad \dot{v}_2 = \frac{F}{m}\cos(\theta) - g; \quad \dot{\theta} = \omega; \quad \dot{\omega} = \frac{1}{I}u_2$$

in the square domain $\mathcal{D} = [-5, 5]^2$. This system admits the planar coordinates p_1, p_2 as linearizing outputs, each with relative degree four. For brevity, we omit the exposition of the 16 residuals related to conditions **CC** and instead focus on the performance of the learned transforms in the downstream tracking task. These results are summarized in Table III.

Metric	Ground Truth	Direct	Cascadic
Avg. error	0.186	0.186	0.186
Final error	0.002	0.002	0.002
Control effort	149.841	149.860	150.488

TABLE III: Tracking performance of the nominal and learned transforms in the planar quadrotor system.

We observe that, despite the increased dimension, a single-network-per-output approach is sufficient for the discovery of an output which can perfectly emulate the analytic nominal linearizing output. However, we would like to further examine the empirical performance of both learned models in settings where linearization cannot be achieved to almost machine precision.

D. Results: Synthetic Systems

To test our proposed algorithm on systems with more complex linearizing outputs, where analytical approaches become intractable, we consider a parameterized class of synthetic systems. In the SISO case, such

systems can be generated by applying a diffeomorphism $\mathbf{z} = \tilde{\Phi}(\mathbf{x})$ and an affine input transformation $\mathbf{u} = \alpha(\mathbf{x}) + \beta(\mathbf{x})\mathbf{v}$ to a linear system in Brunovsky canonical form (2). The resulting dynamics functions are given as: $f(\mathbf{x}) = [D_{\mathbf{x}}\tilde{\Phi}(\mathbf{x})]^{-1}\big(\mathbf{A}\tilde{\Phi}(\mathbf{x}) + \mathbf{B}\alpha(\mathbf{x})\big)$, $g(\mathbf{x}) = [D_{\mathbf{x}}\tilde{\Phi}(\mathbf{x})]^{-1}\mathbf{B}\beta(\mathbf{x})$. One can easily check the linearizability of this system by computing the time derivative of $\tilde{\Phi}(\mathbf{x})$. MIMO systems can be similarly synthesized with a generalized, vector-valued affine control law. To sample a diverse set of such systems, we parameterize the diffeomorphism $\tilde{\Phi}$ as invertible neural networks [24] and randomly sample their weights. More details are provided in Appendix C of the full version [16]. We visualize a representative 2D synthetic system in Figure 2 to illustrate the nonlinearity of the synthesized system.

We consider three system configurations: 2D SISO, 4D SISO, and 8D MIMO systems. The 2D SISO setting allows us to validate the evaluation methodology, providing visual insight into the synthesized systems and the overall tracking performance. The 4D SISO systems demonstrate the advantages of the cascadic approach in the presence of higher-order derivatives. Finally, the 8D MIMO systems illustrate the scalability of the proposed method to higher dimensions. For each setting, we generate 10 synthetic systems. We train and evaluate our learned coordinate transformation on the square domain $\mathcal{D} := [-1.75, 1.75]^n$ using both the linearization quality error (RLQE)–estimated in 100,000 uniformly sampled points–and controller metrics described in Section VI-B. The aggregate results are summarized in Table IV.

1) 2D SISO Systems: For 2D SISO systems, our learned coordinate transformation achieves highly accurate linearization, as evidenced by the RLQE. Consequently, the performance of its corresponding controller closely matches that of the ground-truth controller. Figure 2 illustrates the learned diffeomorphism for a representative system: the box domain $\mathcal{D}$ in the $\mathbf{x}$-space warps nonlinearly in the $\mathbf{z}$-space, while the elliptical Lyapunov function native to the linearized $\mathbf{z}$-space exhibits nonlinear behavior in the $\mathbf{x}$-domain. To contextualize the theoretical results, Figure 1 visualizes the estimated forward-invariant error set in Theorem 2 with estimated learning error bounds. The minimal learning errors in this case lead to very tight error sets. Further numerical results and visualizations for 2D SISO systems can be found in Appendix C of the full version [16].

2) 4D SISO Systems: We compare the direct, single-network approach with our cascadic method on 4D SISO systems. Across 10 randomly generated systems, the cascadic approach achieves an RLQE two orders of magnitude lower than the direct method, effectively mitigating problems arising from high-order differentiations. Furthermore, our cascadic approach is consistently stable and matches the tracking performance of the ground-truth controller. In contrast, the direct approach diverges on *half* of the tested systems (29 out of 640 trajectories across 10 systems). These results establish that a cascaded structure is critical for reliably applying PINN-based methods to complex nonlinear systems.

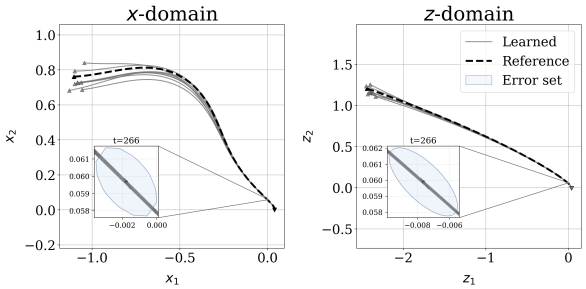


Fig. 1: Visualization of the learned controller executing a reference trajectory from perturbed starting states. The reference goes from top-left to bottom-right. All executed trajectories converge to the reference. We show the estimated forward-invariant error set (Theorem 2) in the insets, where the learning error bounds are estimated via sampling. All trajectories converge to this tight error tube and remain within.

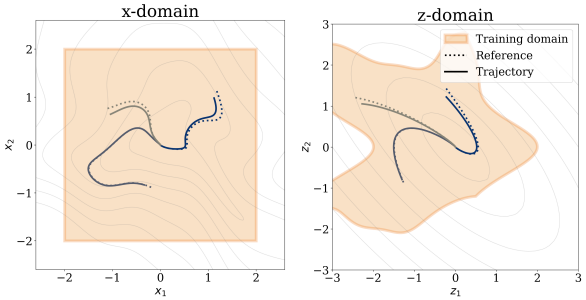


Fig. 2: Sample trajectories for a 2D synthetic SISO system in both actual $\mathbf{x}$-space and linearized $\mathbf{z}$-space.

3) 8D MIMO Systems: Finally, we evaluate our proposed cascadic approach on its ability to generalize to MIMO systems by considering 8D MIMO systems with two linearizing outputs, each with relative degree 4. The increased system dimension, together with the need to learn multiple outputs, introduces additional challenges for the proposed method. However, the learned coordinate transformation again achieves tracking performance on par with the ground-truth controller.

VII. Conclusion

This paper introduced a cascadic physics-informed neural network method to systematically discover feedback-

Metric	2D SISO		4D SISO			8D MIMO	
	Ground Truth	Cascadic	Ground Truth	Direct	Cascadic	Ground Truth	Cascadic
RLQE (↓)	—	0.009%	—	2.92%	0.035%	—	—
Avg. Error (↓)	0.0483	0.0483	0.1172	0.1340	0.1178	0.1079	0.1076
Final Error (↓)	0.0001	0.00001	0.0015	0.0017	0.0020	0.0008	0.0008
Control Effort (↓)	45.58	45.59	103.6850	110.6017	103.4540	183.13	183.5
Perc. Diverged (↓)	—	0.00%	—	4.53%	0.00%	—	0.00%

TABLE IV: Combined tracking performance across 2D SISO, 4D SISO, and 8D MIMO synthetic systems.

linearizing coordinate transformations for nonlinear systems. By decomposing complex, high-order partial differential equations into a sequence of tractable, first-order residual minimizations, our approach successfully mitigated compounding approximation errors and enabled robust downstream tracking control. Extensive validation demonstrated strong empirical performance across real robotic systems and complex synthetic systems of up to eight dimensions. Future work will extend this framework to achieve approximate feedback linearization for systems that are not exactly linearizable [25], [19] and to handle scenarios where the underlying dynamics are subject to uncertainty.

REFERENCES

[1] Alberto Isidori, *Nonlinear control systems: an introduction*, Springer, 1985.

[2] Issa Amadou Tall, "Explicit feedback linearization of control systems," *Proceedings of the 48h IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference*, pp. 7454–7459, 2009.

[3] Maziar Raissi, Paris Perdikaris, and George E Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.

[4] Filippos Fotiadis and Kyriakos G Vamvoudakis, "A physics-informed neural networks framework to solve the infinite-horizon optimal control problem," in *2023 62nd IEEE Conference on Decision and Control (CDC)*. IEEE, 2023, pp. 6014–6019.

[5] Somil Bansal and Claire J Tomlin, "Deepreach: A deep learning approach to high-dimensional reachability," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1817–1824.

[6] A Yeşildirek and Frank L Lewis, "Feedback linearization using neural networks," *Automatica*, vol. 31, no. 11, pp. 1659–1664, 1995.

[7] Savaş Şahin, "Learning Feedback Linearization Using Artificial Neural Networks," *Neural Processing Letters*, vol. 44, no. 3, pp. 625–637, Dec. 2016.

[8] Jonas Umlauft, Thomas Beckers, Melanie Kimmel, and Sandra Hirche, "Feedback linearization using gaussian processes," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 5249–5255.

[9] Tyler Westenbroek, David Fridovich-Keil, Eric Mazumdar, Shreyas Arora, Valmik Prabhu, S Shankar Sastry, and Claire J Tomlin, "Feedback linearization for uncertain systems via reinforcement learning," in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 1364–1371.

[10] Mohamed K Helwa, Adam Heins, and Angela P Schoellig, "Provably robust learning-based approach for high-accuracy tracking control of lagrangian systems," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1587–1594, 2019.

[11] Melissa Greeff and Angela P. Schoellig, "Exploiting Differential Flatness for Robust Learning-Based Tracking Control Using Gaussian Processes," *IEEE Control Systems Letters*, vol. 5, no. 4, pp. 1121–1126, Oct. 2021.

[12] Raktim Gautam Goswami, Prashanth Krishnamurthy, and Farshad Khorrami, "Data-Driven Deep Learning Based Feedback Linearization of Systems with Unknown Dynamics," in *2023 American Control Conference (ACC)*, May 2023, pp. 66–71, ISSN: 2378-5861.

[13] Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz, "Modern Koopman Theory for Dynamical Systems," *SIAM Review*, vol. 64, no. 2, pp. 229–340, May 2022.

[14] Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature Communications*, vol. 9, no. 1, pp. 4950, Nov. 2018.

[15] Shaowu Pan and Karthik Duraisamy, "Physics-Informed Probabilistic Learning of Linear Embeddings of Nonlinear Dynamics with Guaranteed Stability," *SIAM Journal on Applied Dynamical Systems*, vol. 19, no. 1, pp. 480–509, Jan. 2020.

[16] Pavlos Kallinikidis, Fengjun Yang, David Snyder, Jacob Seidman, Nikolai Matni, Paris Perdikaris, and George Pappas, "Physics-informed learning of feedback-linearizing representations," `https://anon-research-3125.github.io/learning-feedback-linearization/`, 2026, Unpublished manuscript. Accessed: 2026-03-31.

[17] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville, "On the spectral bias of neural networks," 2019.

[18] Jesse Bettencourt, Matthew J Johnson, and David Duvenaud, "Taylor-mode automatic differentiation for higher-order derivatives in jax," in *Program transformations for ML workshop at NeurIPS 2019*, 2019.

[19] Tor Arne Johansen and Kenneth J Hunt, "A computational approach to approximate input/state feedback linearization," in *Proceedings of the 39th IEEE Conference on Decision and Control (Cat. No. 00CH37187)*. IEEE, 2000, vol. 5, pp. 4467–4472.

[20] Sifan Wang, Shyam Sankaran, Hanwen Wang, and Paris Perdikaris, "An expert's guide to training physics-informed neural networks," 2023.

[21] Sifan Wang, Yujun Teng, and Paris Perdikaris, "Understanding and mitigating gradient pathologies in physics-informed neural networks," 2020.

[22] Sifan Wang, Hanwen Wang, Jacob H. Seidman, and Paris Perdikaris, "Random weight factorization improves the training of continuous neural representations," 2022.

[23] Jean-Jacques E. Slotine and Weiping Li, *Applied Nonlinear Control*, Prentice Hall, Englewood Cliffs, NJ, 1991.

[24] Lynton Ardizzone, Jakob Kruse, Sebastian Wirkert, Daniel Rahner, Eric W Pellegrini, Ralf S Klessen, Lena Maier-Hein, Carsten Rother, and Ullrich Köthe, "Analyzing inverse problems with invertible neural networks," *arXiv preprint arXiv:1808.04730*, 2018.

[25] Arthur J. Krener, "Approximate linearization by state feedback and coordinate change," *Systems & Control Letters*, vol. 5, no. 3, pp. 181–185, Dec. 1984.

APPENDIX

A. FBL for Multi-Input Multi-Output Systems

Similar to the SISO setting presented in Section III, the conditions for exact feedback linearization of the control affine system (1) in the MIMO setting mirrors can be characterized in terms of the relative degree of the system. Recall that a function $h(x)$ has relative degree r, when, for all $x \in \mathcal{D}$,

$$\mathcal{L}_{g_j}\mathcal{L}_f^i h(\mathbf{x}) = 0 \quad \forall i \in \{0, \ldots, r-2\}, j \in \{1, \ldots, m\}, \tag{17}$$

$$\mathcal{L}_{g_j}\mathcal{L}_f^{r-1} h(\mathbf{x}) \neq 0 \quad \forall j \in \{1, \ldots, m\}. \tag{18}$$

Consider a set of real-valued smooth functions $\{h_i(x)\}_{i=1}^m$, each with relative degree r_i. Denoting $\mathbf{z}_i = h_i(\mathbf{x})$, we have that the linearized state $\mathbf{z} := \left(\mathbf{z}_1, \ldots, \mathbf{z}_m\right)$ evolves as m decoupled chains of integrators (i.e. in Brunovsky canonical form). Specifically, denoting the r-th time derivative of z_i as $h^{(r)}$, we get:

$$\left(\mathbf{z}_1^{(r_1)}, \ldots, \mathbf{z}_m^{(r_m)}\right) = \mathbf{v} := b(\mathbf{x}) + A(\mathbf{x})u, \tag{19}$$

where the drift term $b : \mathbb{R}^n \to \mathbb{R}^m$ and the decoupling matrix $A : \mathbb{R}^n \to \mathbb{R}^{m\times m}$ can be written down analytically, respectively as

$$A(\mathbf{x}) = \begin{pmatrix} \mathcal{L}_{g_1}\mathcal{L}_f^{r_1-1}h_1(\mathbf{x}) & \ldots & \mathcal{L}_{g_m}\mathcal{L}_f^{r_1-1}h_1(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \mathcal{L}_{g_1}\mathcal{L}_f^{r_m-1}h_m(\mathbf{x}) & \ldots & \mathcal{L}_{g_m}\mathcal{L}_f^{r_m-1}h_m(\mathbf{x}) \end{pmatrix}, \tag{20}$$

$$b(\mathbf{x}) = \left(\mathcal{L}_f^{r_1}h_1(\mathbf{x}), \ldots, \mathcal{L}_f^{r_m}h_m(\mathbf{x})\right). \tag{21}$$

Denote the map $\Phi : \mathbb{R}^n \to \mathbb{R}^n$ as:

$$\Phi(\mathbf{x}) = \left(h_1(\mathbf{x}), \ldots, \mathcal{L}_f^{r_1-1}h_1(\mathbf{x}), \ldots, h_m(\mathbf{x}), \ldots, \mathcal{L}_f^{r_m-1}h_m(\mathbf{x})\right). \tag{22}$$

When the decoupling matrix $A(\mathbf{x})$ is invertible and the total relative degree of the system $r := r_1 + \ldots + r_m$ matches the state dimension n, i.e.,

$$r_1 + \ldots + r_m = n, \tag{23}$$

the map Φ is a diffeomorphism and a linearizing coordinate transform for the system (1). The corresponding linearizing control law is given as

$$\mathbf{u} = A(\mathbf{x})^{-1}(\mathbf{v} - b(\mathbf{x})), \tag{24}$$

which is equivalent to the SISO case.

B. Proof of Theorem 2 in the General MIMO setting

We prove Theorem 2 in the MIMO setting, which implies the results in Section V as special cases. Similar to the SISO setting, we consider the task of following a feasible trajectory $(\bar{\mathbf{x}}, \bar{\mathbf{u}})$ and assume that the reference trajectory lies within the training domain.

Assumption 4. For all $t \geq 0$, the reference trajectory lies within the domain, i.e., $\bar{\mathbf{x}}(t) \in \mathcal{D}$, and the reference inputs $\bar{\mathbf{v}}$ are uniformly bounded: $\|\bar{\mathbf{v}}(t)\| \leq \bar{V}$.

Recall that the linearizing coordinate transform takes the form of (22). Therefore, we denote our in learned, *approximate* diffeomorphism $\hat{\Phi}$ as

$$\mathbf{z} = \hat{\Phi}(\mathbf{x}) = \left(\hat{\Phi}_{1,1}(\mathbf{x}), \cdots, \hat{\Phi}_{1,r_1}(\mathbf{x}), \cdots, \hat{\Phi}_{m,1}(\mathbf{x}), \cdots, \hat{\Phi}_{m,r_m}(\mathbf{x})\right)^\top.$$

Mirroring the expressions in (20) and (21), we define

$$A_{\hat{\Phi}}(\mathbf{x}) = \begin{pmatrix} \mathcal{L}_{g_1}\hat{\Phi}_{1,r_1}(\mathbf{x}) & \ldots & \mathcal{L}_{g_m}\hat{\Phi}_{1,r_1}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \mathcal{L}_{g_1}\hat{\Phi}_{m,r_m}(\mathbf{x}) & \ldots & \mathcal{L}_{g_m}\hat{\Phi}_{m,r_m}(\mathbf{x}) \end{pmatrix}, \quad b_{\hat{\Phi}}(\mathbf{x}) = \begin{pmatrix} \mathcal{L}_f\hat{\Phi}_{1,r_1}(\mathbf{x}) \\ \vdots \\ \mathcal{L}_f\hat{\Phi}_{m,r_m}(\mathbf{x}) \end{pmatrix}.$$

The error-feedback control law takes the same form as in the SISO setting:

$$\mathbf{u}(\mathbf{x},t) = -A_{\hat{\Phi}}^{-1}(\mathbf{x})\left(b_{\hat{\Phi}}(\mathbf{x}) - (\bar{\mathbf{v}}(t) + \mathbf{K}(\mathbf{z} - \bar{\mathbf{z}}(t))\right). \tag{25}$$

Here, $\bar{\mathbf{z}} := \hat{\Phi}(\mathbf{x})$, $\bar{\mathbf{v}} := A_{\hat{\Phi}}(\mathbf{x})u + b_{\hat{\Phi}}(\mathbf{x})$, and the robust feedback gain $\mathbf{K}$ is found by solving the $\mathcal{H}_\infty$ Riccati equation (12) with appropriate choices of $\mathbf{Q}$ and $\mathbf{R}$.

To start, we note that under the coordinate transform $\hat{\Phi}$, the *approximately linearized* $\mathbf{z}$ dynamics follows

$$\dot{\mathbf{z}} = \frac{d}{dt}\hat{\Phi}(\mathbf{x}) = \begin{bmatrix} \frac{d}{dt}\hat{\Phi}_{1,1}(\mathbf{x}) \\ \vdots \\ \frac{d}{dt}\hat{\Phi}_{1,r_1-1}(\mathbf{x}) \\ \vdots \end{bmatrix} = \begin{bmatrix} \mathcal{L}_f\hat{\Phi}_{1,1}(\mathbf{x}) \\ \vdots \\ \mathcal{L}_f\hat{\Phi}_{1,r_1-1}(\mathbf{x}) \\ \vdots \end{bmatrix} + \begin{bmatrix} \sum_{i=1}^m \mathcal{L}_{g_i}\hat{\Phi}_{1,1}(\mathbf{x})\mathbf{u}_i \\ \vdots \\ \sum_{i=1}^m \mathcal{L}_{g_i}\hat{\Phi}_{1,r_1-1}(\mathbf{x})\mathbf{u}_i \\ \vdots \end{bmatrix}. \tag{26}$$

To streamline notation, we define the vector function $\mathbf{d}^c : \mathcal{D} \to \mathbb{R}^n$ and matrix function $\mathcal{R} : \mathcal{D} \to \mathbb{R}^{n\times m}$ as

$$\mathbf{d}^c(\mathbf{x}) := \begin{bmatrix} \mathcal{L}_f\hat{\Phi}_{1,1}(\mathbf{x}) - \hat{\Phi}_{1,2}(\mathbf{x}) \\ \vdots \\ \mathcal{L}_f\hat{\Phi}_{1,r_1-1}(\mathbf{x}) - \hat{\Phi}_{1,r_1}(\mathbf{x}) \\ \vdots \end{bmatrix}, \quad [\mathcal{R}(\mathbf{x})]_{i,j} = \mathcal{L}_{g_j}\hat{\Phi}_i(\mathbf{x}).$$

Applying the control (25), the $\mathbf{z}$ dynamics (26) evolves as

$$\begin{aligned} \dot{\mathbf{z}} &= (\mathbf{A}\mathbf{z} + \mathbf{B}\mathbf{v} + \mathbf{d}^c(\mathbf{x})) + \mathcal{R}(\mathbf{x})\mathbf{u} \\ &= (\mathbf{A}\mathbf{z} + \mathbf{B}\mathbf{v} + \mathbf{d}^c(\mathbf{x})) + \mathcal{R}(\mathbf{x})A_{\hat{\Phi}}^{-1}(\mathbf{x})\left(\mathbf{v} - b_{\hat{\Phi}}(\mathbf{x})\right) \\ &= \mathbf{A}\mathbf{z} + \left(\mathbf{B} + \underbrace{\mathcal{R}(\mathbf{x})A^{-1}(\mathbf{x})}_{=:\Delta_B(\mathbf{x})}\right)\mathbf{v} + \left(\underbrace{\mathbf{d}^c(\mathbf{x}) - \mathcal{R}(\mathbf{x})A_{\hat{\Phi}}^{-1}(\mathbf{x})b_{\hat{\Phi}}(\mathbf{x})}_{=:\mathbf{d}^z(\mathbf{x})}\right). \end{aligned} \tag{27}$$

Define the $\mathbf{z}$-state error as $\mathbf{e} := \mathbf{z} - \bar{\mathbf{z}}$. The error dynamics evolve as

$$\dot{\mathbf{e}} = \mathbf{A}\mathbf{e} + (\mathbf{B} + \mathbf{\Delta}_B(\mathbf{x}))\mathbf{K}\mathbf{e} + \underbrace{(\mathbf{\Delta}_B(\mathbf{x}) - \mathbf{\Delta}_B(\bar{\mathbf{x}}))\bar{\mathbf{v}} + \mathbf{d}^z(\mathbf{x}) - \mathbf{d}^z(\bar{\mathbf{x}})}_{=:\mathbf{d}}.$$

We assume that the error terms Δ_B and $\mathbf{d}^z$ are small when the models are well-trained on the domain.

Assumption 5. We assume that for all $\mathbf{x} \in \mathcal{D}$, the error terms Δ_B and $\mathbf{d}^z$ are bounded:

$$\|\Delta_B(\mathbf{x})\|_{2\to 2} \le \bar{\delta}, \quad \|\mathbf{d}^z(\mathbf{x})\|_2 \le \bar{d}^z.$$

Note that from Assumption 5, we clearly have that

$$\|\mathbf{d}\| \le \|\Delta_B(\mathbf{x}) + \Delta_B(\bar{\mathbf{x}})\|\,\|\bar{\mathbf{v}}\| + \|\mathbf{d}^z(\mathbf{x}) - \mathbf{d}^z(\bar{\mathbf{z}})\| \le 2(\bar{\delta}\bar{V} + \bar{d}^z) =: \bar{d}.$$

The boundedness of the approximation errors and the reference signal allows us to establish a forward-invariant set of the $\mathbf{z}$ tracking error.

Lemma 3. *Under Assumptions 4 and 5, if $\bar{\delta} < \frac{\sqrt{\lambda_{min}(\mathbf{R})}}{\gamma}$ and if $\mathbf{x}(t) \in \mathcal{D}$ for all $t \ge 0$, the tracking error $\mathbf{e}$ is uniformly ultimately bounded by the sublevel set*

$$\Omega_z = \left\{ \mathbf{e} \mid \mathbf{e}^\top\mathbf{P}\mathbf{e} \le \frac{\lambda_{max}(\mathbf{P})\bar{d}^2}{\lambda_{min}(\mathbf{Q})\left(\frac{1}{\gamma^2} - \frac{\bar{\delta}^2}{\lambda_{min}(\mathbf{R})}\right)} \right\}. \tag{28}$$

Proof. Consider the Lyapunov function for the error dynamics

$$V(\mathbf{e}) = \frac{1}{2}\mathbf{e}^\top \mathbf{P}\mathbf{e}, \tag{29}$$

we have that

$$\begin{aligned}
\dot{V}(\mathbf{e}) &= \frac{1}{2}\left[\mathbf{e}^\top \mathbf{P}(\mathbf{A} + (\mathbf{B} + \Delta_B)\mathbf{K}\mathbf{e} + \mathbf{d}) + (\mathbf{A} + (\mathbf{B} + \Delta_B)\mathbf{K}\mathbf{e} + \mathbf{d})^\top \mathbf{P}\mathbf{e}\right] \\
&= \frac{1}{2}\mathbf{e}^\top(\mathbf{P}\mathbf{A} + \mathbf{A}^\top\mathbf{P} - 2\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P})\mathbf{e} + \frac{1}{2}\mathbf{e}^\top(\mathbf{P}\Delta_\mathbf{B}\mathbf{K} + \mathbf{K}^\top\Delta_B^\top\mathbf{P})\mathbf{e} + \mathbf{e}^\top\mathbf{P}\mathbf{d} \\
&= \frac{1}{2}\mathbf{e}^\top(-\mathbf{Q} - \frac{\mathbf{P}\mathbf{P}}{\gamma^2} - \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P})\mathbf{e} - \frac{1}{2}\mathbf{e}^\top(\mathbf{P}\Delta_\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\Delta_B^\top\mathbf{P})\mathbf{e} + \mathbf{e}^\top\mathbf{P}\mathbf{d} \\
&= -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2\gamma^2}\mathbf{e}^\top\mathbf{P}\mathbf{P}\mathbf{e} - \frac{1}{2}\underbrace{\mathbf{e}^\top\left(\mathbf{P}\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P} + \mathbf{P}\Delta_B\mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P} + \mathbf{P}\mathbf{B}\mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\right)\mathbf{e}}_{(\triangle)} + \mathbf{e}^\top\mathbf{P}\mathbf{d}.
\end{aligned} \tag{30}$$

Denoting $\mathbf{w} := \mathbf{R}^{-1}\mathbf{B}^\top\mathbf{P}\mathbf{e}$,

$$\begin{aligned}
(\triangle) &= \mathbf{w}^\top\mathbf{R}\mathbf{w} + (\mathbf{P}\mathbf{e})^\top\Delta_B\mathbf{w} + \mathbf{w}^\top\Delta_B^\top\mathbf{P}\mathbf{e} \\
&= (\mathbf{w} + \mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\mathbf{e})^\top\mathbf{R}(\mathbf{w} + \mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\mathbf{e}) - (\mathbf{P}\mathbf{e})^\top\Delta_B\mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\mathbf{e} \\
&\geq -(\mathbf{P}\mathbf{e})^\top\Delta_B\mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\mathbf{e}.
\end{aligned}$$

As a result, we can bound $\dot{V}$ as

$$\begin{aligned}
\dot{V}(\mathbf{e}) &\leq -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2\gamma^2}\mathbf{e}^\top\mathbf{P}\mathbf{P}\mathbf{e} + \frac{1}{2}(\mathbf{P}\mathbf{e})^\top\Delta_B\mathbf{R}^{-1}\Delta_B^\top\mathbf{P}\mathbf{e} + \mathbf{e}^\top\mathbf{P}\mathbf{d} \\
&= -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2}\mathbf{e}^\top\mathbf{P}\left(\frac{1}{\gamma^2}I - \Delta_B\mathbf{R}^{-1}\Delta_B^\top\right)\mathbf{P}\mathbf{e} + \mathbf{d}^\top\mathbf{P}\mathbf{e} \\
&\leq -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2}\lambda_{\min}\left(\frac{1}{\gamma^2}I - \Delta_B\mathbf{R}^{-1}\Delta_B^\top\right)\|\mathbf{P}\mathbf{e}\|^2 + \bar{d}\,\|\mathbf{P}\mathbf{e}\|
\end{aligned}$$

The relevant minimum eigenvalue is bounded as

$$\lambda_{\min}\left(\frac{1}{\gamma^2}I - \Delta_B\mathbf{R}^{-1}\Delta_B^\top\right) = \frac{1}{\gamma^2} - \lambda_{\max}\left(\Delta_B\mathbf{R}^{-1}\Delta_B^\top\right) \geq \underbrace{\frac{1}{\gamma^2} - \frac{\bar{\delta}^2}{\lambda_{\min}(\mathbf{R})}}_{=:\eta} > 0,$$

where the last inequality follows from our assumption that $\bar{\delta} < \sqrt{\lambda_{\min}(\mathbf{R})}/\gamma$. Denoting this lower bound as η, completing the square on $\|\mathbf{P}\mathbf{e}\|$ yields

$$\begin{aligned}
\dot{V}(\mathbf{e}) &\leq -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2}\eta\,\|\mathbf{P}\mathbf{e}\|^2 + \bar{d}\,\|\mathbf{P}\mathbf{e}\| \\
&\leq -\frac{1}{2}\mathbf{e}^\top\mathbf{Q}\mathbf{e} - \frac{1}{2}\left(\sqrt{\eta}\,\|\mathbf{P}\mathbf{e}\| - \frac{\bar{d}}{\sqrt{\eta}}\right)^2 + \frac{\bar{d}^2}{2\eta} \\
&\leq -\frac{1}{2}\lambda_{\min}(\mathbf{Q})\,\|\mathbf{e}\|^2 + \frac{\bar{d}^2}{2\eta}
\end{aligned}$$

As a result, the Lyapunov function is strictly decreasing when $\|\mathbf{e}\|^2 > \frac{\bar{d}^2}{\lambda_{\min}(\mathbf{Q})\eta}$. The tracking error is thus uniformly ultimately bounded by the sublevel set (28). □

To connect the error of linearized states $\mathbf{z}$, to that of the physical states $\mathbf{x}$, we leverage Assumption 1, which states that the inverse of the diffeomorphism $\hat{\Phi}^{-1}$ is well-conditioned. This leads to the following result.

Lemma 4. *Under the assumptions of Lemma 3 and Assumption 1, the* $\mathbf{x}$ *tracking error* $\mathbf{e}_x := \mathbf{x} - \bar{\mathbf{x}}$ *is*

uniformly ultimately bounded by the set

$$\Omega_x = \left\{ \mathbf{e} \mid \|\mathbf{e}_x\|^2 \leq \frac{\kappa(\mathbf{P})\bar{d}^2}{l_{\hat{\Phi}}^2 \lambda_{min}(\mathbf{Q}) \left(\frac{1}{\gamma^2} - \frac{\bar{\delta}^2}{\lambda_{min}(\mathbf{R})} \right)} \right\},$$

where $\kappa(\cdot)$ denotes the condition number of the given matrix.

Proof. From Lemma 3, there exists a time T, independent of starting time t_0, such that for $t \geq T$,

$$(\mathbf{z} - \bar{\mathbf{z}})^\top \mathbf{P} (\mathbf{z} - \bar{\mathbf{z}}) \leq \frac{\lambda_{\max}(\mathbf{P})\bar{d}^2}{\lambda_{\min}(\mathbf{Q}) \left(\frac{1}{\gamma^2} - \frac{\bar{\delta}^2}{\lambda_{\min}(\mathbf{R})} \right)}.$$

Thus, for the same time T, we have that

$$\begin{aligned} \|\mathbf{e}_x\|^2 &= \|\mathbf{x} - \bar{\mathbf{x}}\|^2 \\ &\leq \frac{1}{l_{\hat{\Phi}}^2} \left\| \hat{\Phi}(\mathbf{x}) - \hat{\Phi}(\bar{\mathbf{x}}) \right\|^2 \\ &\leq \frac{1}{l_{\hat{\Phi}}^2 \lambda_{\min}(\mathbf{P})} \mathbf{e}_z^\top \mathbf{P} \mathbf{e}_z \\ &\leq \frac{1}{l_{\hat{\Phi}}^2 \lambda_{\min}(\mathbf{P})} \frac{\lambda_{\max}(\mathbf{P})\bar{d}^2}{\lambda_{\min}(\mathbf{Q}) \left(\frac{1}{\gamma^2} - \frac{\bar{\delta}^2}{\lambda_{\min}(\mathbf{R})} \right)}. \end{aligned}$$

□

Finally, the above results explicitly assume that $\mathbf{x}$ does not leave the domain $\mathcal{D}$, where the approximation quality is guaranteed. We thus leverage the forward invariant error set to robustify our assumption on our reference trajectory.

Assumption 6. We assume that the reference trajectory states lie within the domain, i.e.,

$$\bar{\mathbf{x}}(t) \in \mathcal{D} \ominus \Omega_x, \quad \forall t \geq 0,$$

and that the reference inputs $\bar{v}$ is uniformly bounded:

$$\|\bar{v}(t)\| \leq \bar{V}, \quad \forall t \geq 0.$$

Here $\ominus$ denotes the Minkowski difference.

Theorem 5. *Under assumptions 5, 1, and 6, if $\bar{\delta} < \frac{\sqrt{\lambda_{min}(\mathbf{R})}}{\gamma}$ and $\mathbf{e}_x(0) \in \Omega_x$, the tracking error $\mathbf{e}_x$ will be uniformly ultimately bounded by the sublevel set Ω_x.*

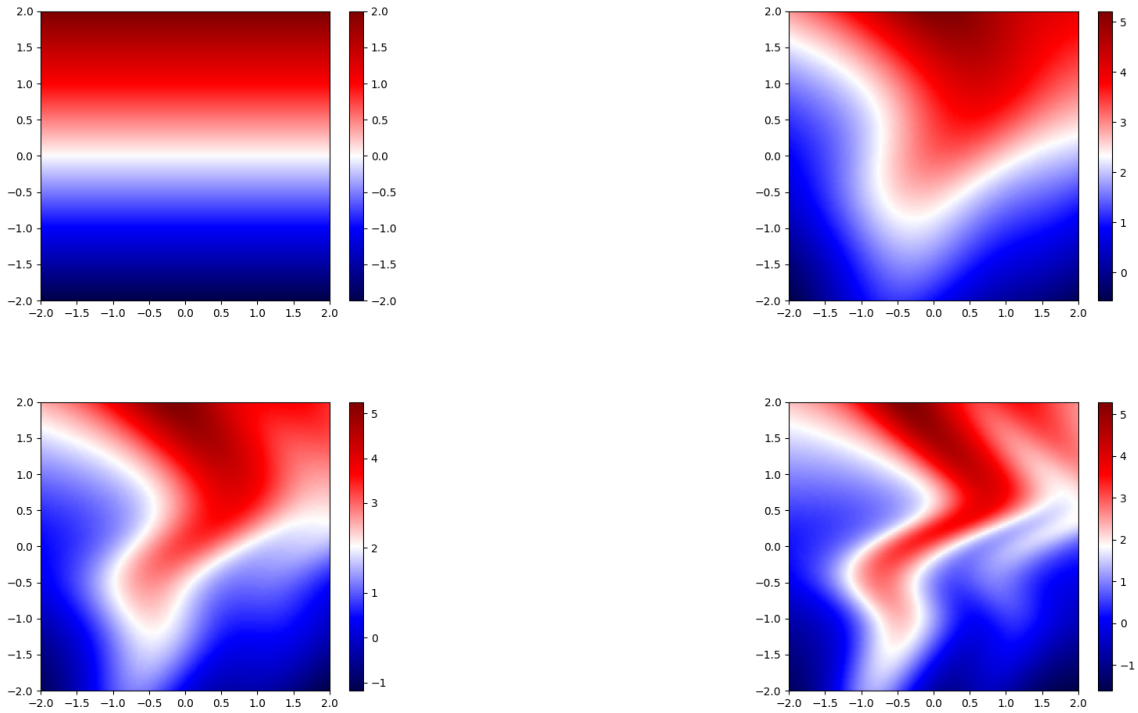
Proof. Following the assumptions, the state trajectory always lies within $\mathcal{D}$. The rest follows directly from Lemma 4. □

C. System synthesis details

As already described in Section VI-D, we synthesize exactly feedback linearizable systems via randomly generated diffeomorphisms building on a bidirectional affine coupling mechanism. After splitting the input $\mathbf{x}$ to $\mathbf{x}_1, \mathbf{x}_2$, the diffeomorphism is defined as:

$$\begin{aligned} \mathbf{y}_1 &= \mathbf{x}_1 + t_1(\mathbf{x}_2) \\ \mathbf{y}_2 &= \mathbf{x}_2 + t_2(\mathbf{y}_1) \end{aligned}$$

and t_1, t_2 are parameterized as a linear combination of Gaussian kernel functions. In our experiments, we fix $s_1(\mathbf{x}) = s_2(\mathbf{x}) = \mathbf{1}$ and use Gaussian kernels to parameterize t_1 and t_2. Analytically, we define each transformation as $t_i(\mathbf{x}) := \mathbf{W}_i \phi_i(\mathbf{x})$, where $\mathbf{W}_i \in \mathbb{R}^{d \times (n/2)}$ is a weight matrix and $\phi_i(\mathbf{x}) \in \mathbb{R}^d$ is a vector of Gaussian basis functions with components $\phi_{i,k}(\mathbf{x}) := \exp\left(-\frac{1}{2} (\mathbf{x} - \mathbf{c}_{i,k})^\top \mathbf{\Sigma}_{i,k}^{-1} (\mathbf{x} - \mathbf{c}_{i,k}) \right)$ for $k = 1, \ldots, d$. Here, $\mathbf{c}_{i,k} \in \mathbb{R}^{n/2}$



(c) $\mathbf{\Sigma}_k \sim \mathcal{U}(0, 2)$ (d) $\mathbf{\Sigma}_k \sim \mathcal{U}(0, 4)$

Fig. 3: A linear and representative nominal linearizing outputs for different scales of Σ_k and fixed W_k, c_k for the synthetic systems.

	Num. Kernels	Σ_k	c_k	W_k
t_1	32	$\mathcal{U}(0, 2)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$
t_2	32	$\mathcal{U}(0, 2)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$

TABLE V: Kernel configuration for $t_1(x)$ and $t_2(x)$ in the synthetic 2D systems of the main paper.

are the kernel centers and $\mathbf{\Sigma}_{i,k} \in \mathbb{R}^{(n/2)\times(n/2)}$ are diagonal covariance matrices. Thus, $t_1(\mathbf{x})$ and $t_2(\mathbf{x})$ are linear combinations of Gaussian kernels. We generate diverse invertible transforms by sampling the nonzero parameters of $t_i(\mathbf{x})$ uniformly. The smoothness-and, consequently, learning task difficulty—is controlled by the upper bound on the diagonal covariance entries. The exact parameters used for the synthetic experiments of Section VI-D are summarized in Tables V, VI and VII.

To demonstrate the applicability of the proposed method to control affine systems that are characterized by highly non-regular vector fields f, g and are feedback linearized from less smooth diffeomorphisms as the one in Figure 3d, we additionally provide numerical results for the configuration in Table VIII. Representative vector fields are presented in Figure 5, and, following the evaluation methodology of the main paper, we provide the numerical results in Table IX. We observe that, even in this more challenging setup, only one out of ten systems exhibits unstable behavior, despite the absence of dedicated hyperparameter tuning during training.

	Num. Kernels	Σ_k	c_k	W_k
t_1	32	$\mathcal{U}(0, 2)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$
t_2	32	$\mathcal{U}(0, 1)$	$\mathcal{U}(-0.8, 0.8)$	$\mathcal{U}(-1, 1)$

TABLE VI: Kernel configuration for $t_1(x)$ and $t_2(x)$ in the synthetic 4D systems of the main paper.

	Num. Kernels	Σ_k	c_k	W_k
t_1	16	$\mathcal{U}(0, 0.6)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$
t_2	16	$\mathcal{U}(0, 0.2)$	$\mathcal{U}(-0.6, 0.6)$	$\mathcal{U}(-1, 1)$

TABLE VII: Kernel configuration for $t_1(x)$ and $t_2(x)$ in the synthetic 8D systems of the main paper.

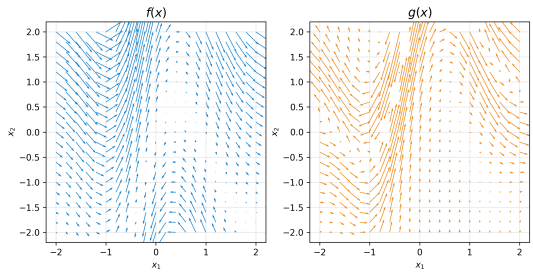


Fig. 4: Sample vector fields f, g of the 2D SISO synthetic systems generated from the configuration of Table V.

	Num. Kernels	Σ_k	c_k	W_k
t_1	32	$\mathcal{U}(0, 4)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$
t_2	32	$\mathcal{U}(0, 4)$	$\mathcal{U}(-1, 1)$	$\mathcal{U}(-1, 1)$

TABLE VIII: Kernel configuration for $t_1(x)$ and $t_2(x)$ in the synthetic non-smooth 2D systems.

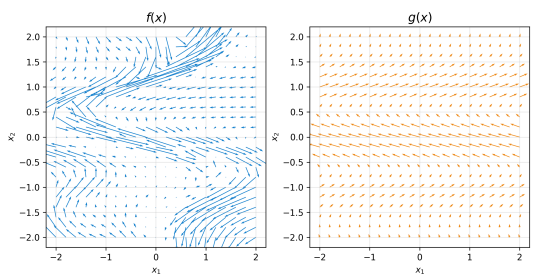


Fig. 5: Sample vector fields f, g of the 2D SISO synthetic systems generated from the configuration of Table VIII.

Metric	**Ground Truth**	**Learned**
RLQE	—	$\mathbf{2.86 \times 10^{-4}}$
Avg. error	0.1177	0.1178
Final error	0.0002	0.0002
Control effort	42.70	42.74
Diverged (any)	—	0.1000
Perc. diverged	—	0.00156

TABLE IX: Tracking performance in 2D SISO systems generated by from the configuration of Table VIII.